

Homework 7

- (i) a) $OPT(i, j)$ = maximum coins obtainable by bursting all balloons between index i and j , exclusive of i and j .

We'll assume balloon i is to the left and balloon j is to the right.

Only balloons in the range (i, j) are to be burst.

Add two virtual balloons with value 1 to both ends of $nums$.

$nums[-1] = 1$, $nums[n] = 1$

so the size is $n + 2$, A = size of $n + 2$

- b) If k is the last balloon burst between i and j , the coins gained are.

$$A[i] \times A[k] \times A[j]$$

plus the coins from the left and right subproblems.

$$OPT(i, j) = \max_{k \in (i, j)} [OPT(i, k) + OPT(k, j) + A[i] \times A[k] \times A[j]]$$

- c) Base case :

$$OPT(i, j) = 0 \text{ if } j \leq i + 1 \text{ (no balloons in between)}$$

final answer is $OPT(0, n+1)$

Pseudocode:

function solve_balloons(nums):

$n = \text{length}(\text{nums})$

$A = [i] + \text{nums} + [i]$

$N = n + 2$

memo = array $[N][N]$ filled with 0

for length from 2 to $N-1$:

for i from 0 to $N-\text{length}-1$:

$j = i + \text{length}$

for k from $i+1$ to $j-1$:

coins = memo[i][k] + memo[k][j]

+ A[i] * A[k] * A[j]

memo[i][j] = min(memo[i][j], coins)

return memo[0][N+1]

d) Time complexity:

Three ~~are~~ nested loops

$$O(N^3) = O(n^3)$$

It is efficient, much faster than brute force $O(n!)$

② a) $\text{OPT}(i, j)$ = maximum score difference
current player can achieve over
the opponent, given that the only
marbles remaining are from index
 i to j inclusive.

b) Current player has 2 choices:

1. Pick the leftmost marble $m[i]$:

$$m[i] - \text{OPT}(i+1, j)$$

2. Pick the rightmost marble $m[j]$:

$$m[j] - \text{OPT}(i, j-1)$$

Recurrence formula:

$$\text{OPT}(i, j) = \max(m[i] - \text{OPT}(i+1, j), m[j] - \text{OPT}(i, j-1))$$

c) Pseudocode:

Base case: When there's only one
marble ($i = j$)

$$\text{OPT}(i, j) = m[i]$$

function solve_marbles(m):

$N = \text{length}(m)$

memo = new 2D array of size $N \times N$

for i from 0 to $N-1$:

$$\text{memo}[i][i] = m[i]$$

for length from 2 to N:
 for i from 0 to N-length:

for j = i + length - 1

take_left = m[i] + memo[i+1][j]

take_right = m[j] + memo[i][j-1]

memo[i][j] = max(take_left, take_right)

return memo[0][N-1]

d) Time complexity:
 Outer loop: length = 2..N
 $\therefore O(N)$

Inner loop: i = 0..N-length
 $\therefore O(N)$

Each OPT(i, j) computation is $O(1)$

Total time complexity $\therefore O(N^2)$
 It is efficient.

③ a) We need the longest mountain subsequence increasing then decreasing.

$LIS[i]$ = length of longest increasing subsequence ending at i

$LDS[i]$ = length of longest decreasing subsequence starting at i

The best mountain peaking at i has length:
 $LIS[i] + LDS[i] - 1$

b) Recurrence relation

$$LIS[i] = 1 + \max_{\substack{j < i, \\ H[j] < H[i]}} LIS[j]$$

$$LDS[i] = 1 + \max_{\substack{j > i, \\ H[j] < H[i]}} LDS[j]$$

Base case: $LIS[i] = LDS[i] = 1$

final answer:

$$\text{min removed} = n - \max(LIS[i] + LDS[i] - 1)$$

c) Pseudocode

function solve_band-formation (H):

$n = \text{len}(H)$

$LIS = [1] * n$

$LDS = [1] * n$

 for i in range(n):

 for j in range(i):

 if $H[j] < H[i]$:

$LIS[i] = \max(LIS[i], LIS[j] + 1)$

 for i in range($n-1, -1, -1$):

 for j in range($i+1, n$):

 if $H[j] < H[i]$:

$LDS[i] = \max(LDS[i], LDS[j] + 1)$

$\text{max_len} = \max(LIS[i] + LDS[i] - 1$

 for i in range(n))

 return $n - \text{max_len}$

d) Time complexity:

(i.e. Both LIS and LDS use nested loops
 $\Rightarrow O(n^2)$ total time.

It is efficient.